

Sorting and Searching

Part One

Outline for Today

- ***Searching a List***
 - Finding what you're looking for.
- ***Sorting Algorithms***
 - How quickly can we get things in order?

Recap from Last Time

```
double averageOf(const Vector<int>& vec) {  
    double total = 0.0;  
  
    for (int i = 0; i < vec.size(); i++) {  
        total += vec[i];  
    }  
  
    return total / vec.size();  
}
```

Assume any individual statement takes one unit of time to execute. If the input Vector has n elements, how many time units will this code take to run?

```
double averageOf(const Vector<int>& vec) {
```

```
1 double total = 0.0;
```

```
    for (int i = 0; i < vec.size(); i++) {  
        total += vec[i];  
    }
```

```
    return total / vec.size();  
}
```

Assume any individual statement takes one unit of time to execute. If the input Vector has n elements, how many time units will this code take to run?

```
double averageOf(const Vector<int>& vec) {
```

```
1 double total = 0.0;
```

```
    for (int i = 0; i < vec.size(); i++) {  
        total += vec[i];  
    }
```

```
    return total / vec.size();  
}
```

One possible answer: $3n + 4$.

```
double averageOf(const Vector<int>& vec) {
```

```
1 double total = 0.0;
```

```
    for (int i = 0; i < vec.size(); i++) {  
        total += vec[i];  
    }
```

```
    return total / vec.size();  
}
```

~~One possible answer: $3n + 4$.~~

More useful answer: **$O(n)$** .

```
void printStars(int n) {  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            cout << '*' << endl;  
        }  
    }  
}
```

Work Done: **$O(n^2)$** .

New Stuff!

Searching a List

598	356	772	667	347	800	173	795	842	663	474	277	490
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

I'm going to show you a list of numbers.
Does it contain 137?

Linear Search

- This approach of scanning all n items of a list, one at a time, is called *linear search*.
- If the item you're looking for isn't there, or if it's at the far end of the list, it takes time $O(n)$.
 - There are n items in the list.
 - We have to look at each one of them to see if it's the one we're looking for.
- This is fine for small n , but imagine you're working at large scale (e.g. customer records, satellite imagery, etc.)
- Can we do better?

What if the numbers are
sorted from low to high?

2	44	45	46	49	50	51	52	54	55
---	----	----	----	----	----	----	----	----	----



I'm going to show you a list of numbers.
Does it contain 54?

2	44	45	46	49	50	51	52	54	55
---	----	----	----	----	----	----	----	----	----



I'm going to show you a list of numbers.
Does it contain 54?

33	38	39	40	41	42	44	45	46	49	5
----	----	----	----	----	----	----	----	----	----	---



I'm going to show you a list of numbers.
Does it contain 54?

33	38	39	40	41	42	44	45	46	47
----	----	----	----	----	----	----	----	----	----



I'm going to show you a list of numbers.
Does it contain 54?

33	38	39	40	41	42	44	45	46	49	50	51	52	54	55
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----



I'm going to show you a list of numbers.
Does it contain 54?

46	49	50	51	52	54	55
----	----	----	----	----	----	----

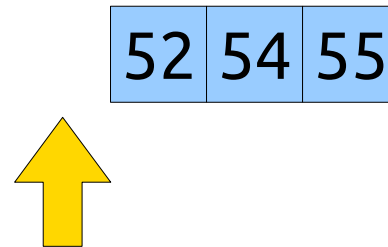


I'm going to show you a list of numbers.
Does it contain 54?

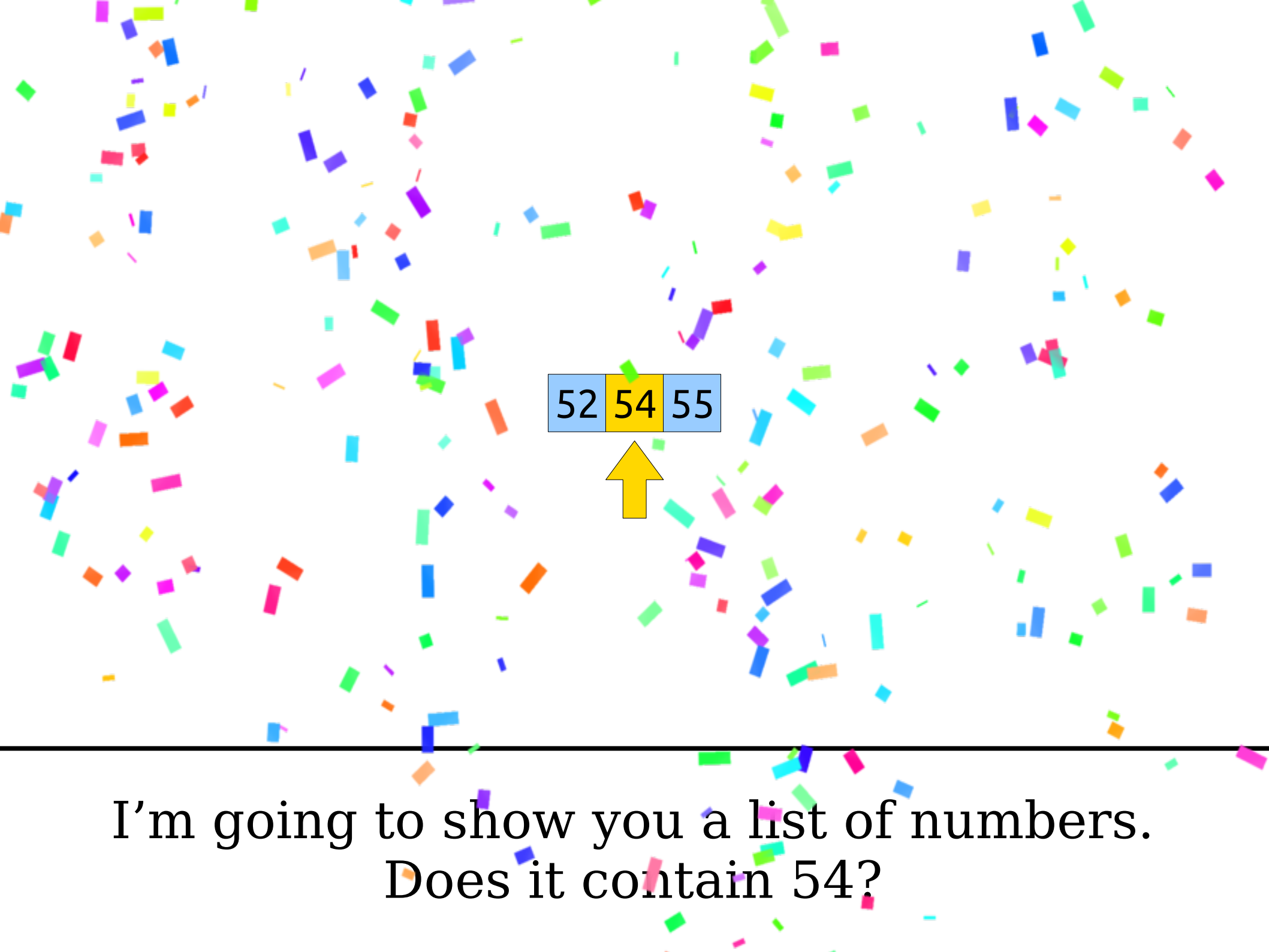
46	49	50	51	52	54	55
----	----	----	----	----	----	----



I'm going to show you a list of numbers.
Does it contain 54?



I'm going to show you a list of numbers.
Does it contain 54?



52	54	55
----	----	----



I'm going to show you a list of numbers.
Does it contain 54?

Binary Search

- ***Binary search*** is a very fast algorithm for searching a sorted list for a particular item.
- ***Base Case:*** If the list is empty, your item isn't there.
- ***Recursive Case:*** Look at the middle element of the list. Then...
 - ... if it's your element, you're done!
 - ... if it's bigger than your element, recursively search the elements to the left.
 - ... if it's smaller than your element, recursively search the elements to the right.

```

bool binarySearchRec(const Vector<int>& elems, int key,
                    int low, int high) {
    /* Base case: If we're out of elements, horror of horrors!
     * Our element does not exist.
     */
    if (low == high) return false;

    /* Look at just the the middle element. Compute its index. */
    int mid = low + (high - low) / 2;

    /* We might find what we're looking for! */
    if (key == elems[mid]) return true;

    /* Otherwise, discard half the elements and search
     * the appropriate section.
     */
    if (key < elems[mid]) {
        return binarySearchRec(elems, key, low, mid);
    } else {
        return binarySearchRec(elems, key, mid + 1, high);
    }
}

bool binarySearch(const Vector<int>& elems, int key) {
    return binarySearchRec(elems, key, 0, elems.size());
}

```


How Fast is Binary Search?

33	38	39	40	41	42	44	45	46	49	50	51	52	54	55
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Suppose we begin with an array of n elements.

Each round throws away (roughly) half the array elements.

After r rounds, there are (roughly) $n / 2^r$ elements left.

The algorithm stops after we're down to one element (or at least, right after that.).

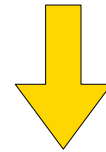
So we need to solve for the value of r where $n / 2^r = 1$.

$$n / 2^r = 1$$

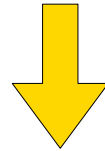
$$n = 2^r$$

$$\log_2 n = r$$

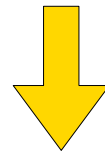
So binary search finishes in **$O(\log n)$** rounds.



46	49	50	51	52	54	55
----	----	----	----	----	----	----



46	49	50
----	----	----



46

What value of r solves this equation?

Answer at

<https://cs106b.stanford.edu/pollev>

Fun Facts About Logarithms

- ***Great Intuition:*** $\log_2 n$ is the number of times you can divide n by two before it drops to 1.
 - More generally, you can only divide n by k roughly $\log_k n$ times before it drops to 1.
- Logarithms grow *extremely* slowly.
 - $\log_2 1,000 \approx ??$.

Fun Facts About Logarithms

- ***Great Intuition:*** $\log_2 n$ is the number of times you can divide n by two before it drops to 1.
 - More generally, you can only divide n by k roughly $\log_k n$ times before it drops to 1.
- Logarithms grow *extremely* slowly.
 - $\log_2 1,000 \approx 10$.
 - $\log_2 1,000,000 \approx ??$.

Fun Facts About Logarithms

- ***Great Intuition:*** $\log_2 n$ is the number of times you can divide n by two before it drops to 1.
 - More generally, you can only divide n by k roughly $\log_k n$ times before it drops to 1.
- Logarithms grow *extremely* slowly.
 - $\log_2 1,000 \approx 10$.
 - $\log_2 1,000,000 \approx 20$.
 - $\log_2 1,000,000,000 \approx ??$.

Fun Facts About Logarithms

- ***Great Intuition:*** $\log_2 n$ is the number of times you can divide n by two before it drops to 1.
 - More generally, you can only divide n by k roughly $\log_k n$ times before it drops to 1.
- Logarithms grow *extremely* slowly.
 - $\log_2 1,000 \approx 10$.
 - $\log_2 1,000,000 \approx 20$.
 - $\log_2 1,000,000,000 \approx 30$.
 - $\log_2 (\# \text{ protons in the observable universe}) \approx \text{??}$.

Fun Facts About Logarithms

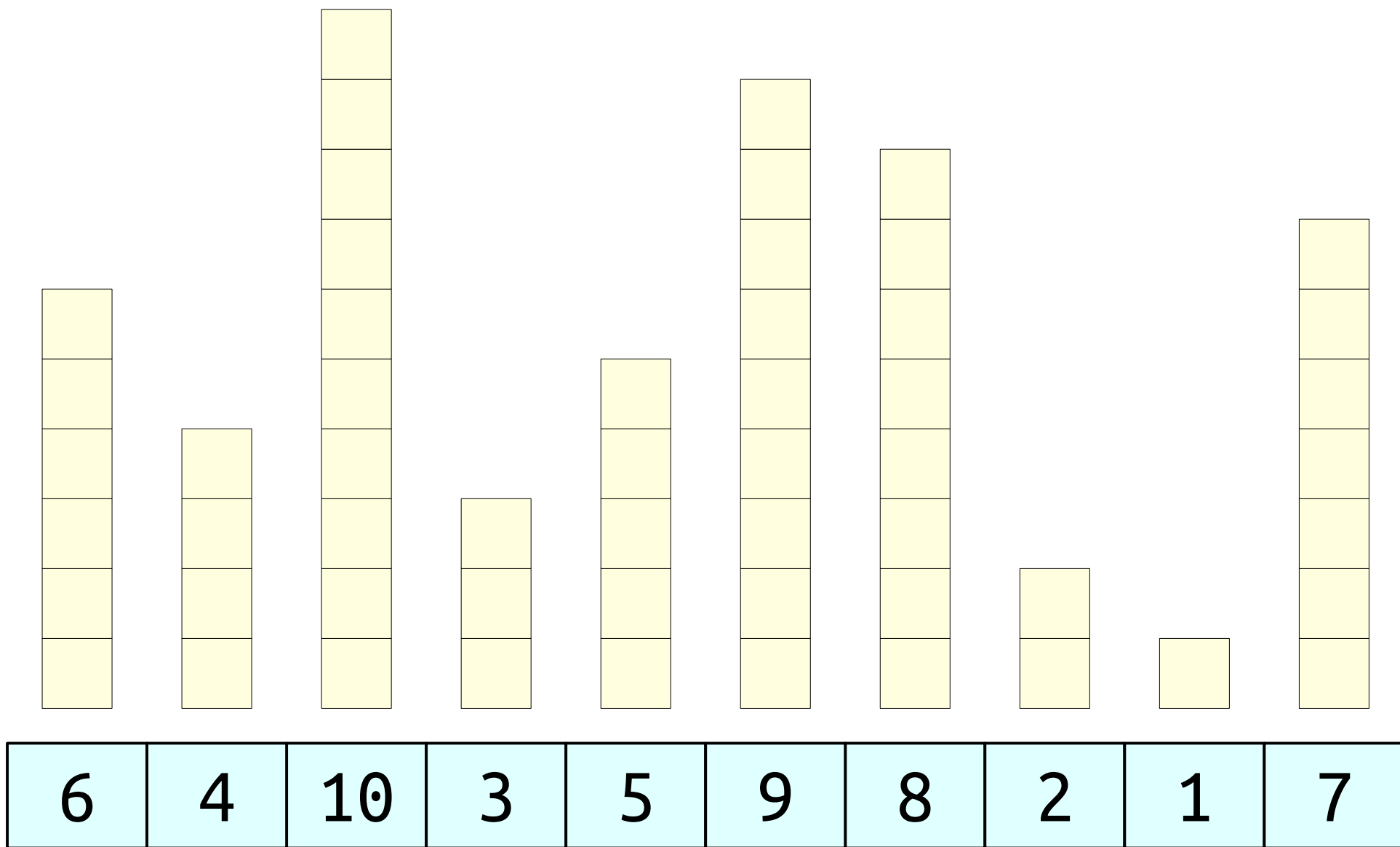
- ***Great Intuition:*** $\log_2 n$ is the number of times you can divide n by two before it drops to 1.
 - More generally, you can only divide n by k roughly $\log_k n$ times before it drops to 1.
- Logarithms grow *extremely* slowly.
 - $\log_2 1,000 \approx 10$.
 - $\log_2 1,000,000 \approx 20$.
 - $\log_2 1,000,000,000 \approx 30$.
 - $\log_2 (\# \text{ protons in the observable universe}) \approx 300$.
- In big-O notation, we don't write log bases and just say $O(\log n)$.
 - Why? *Hint:* big-O go “om nom nom” to constant factors.

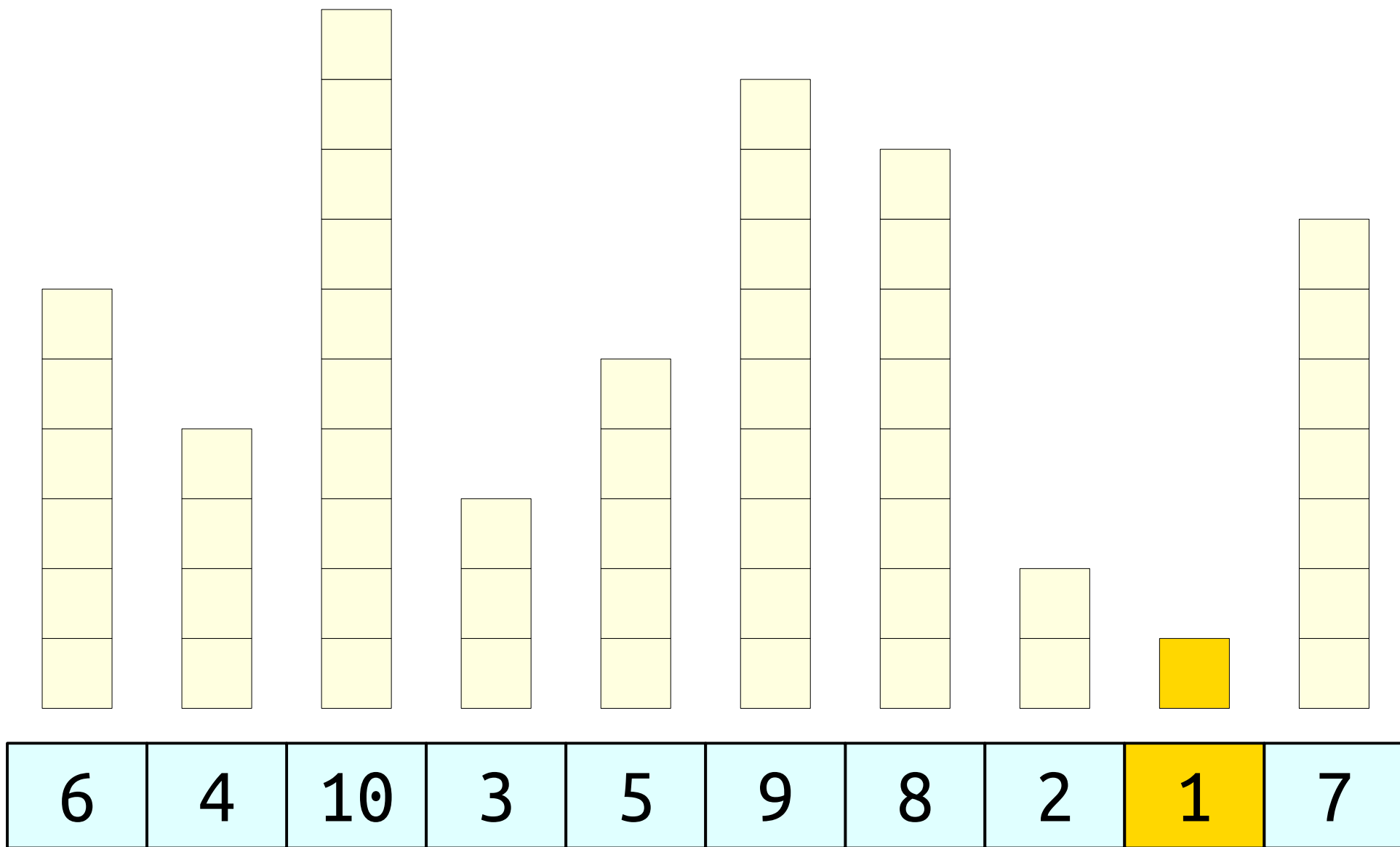
How do you sort a list of elements?

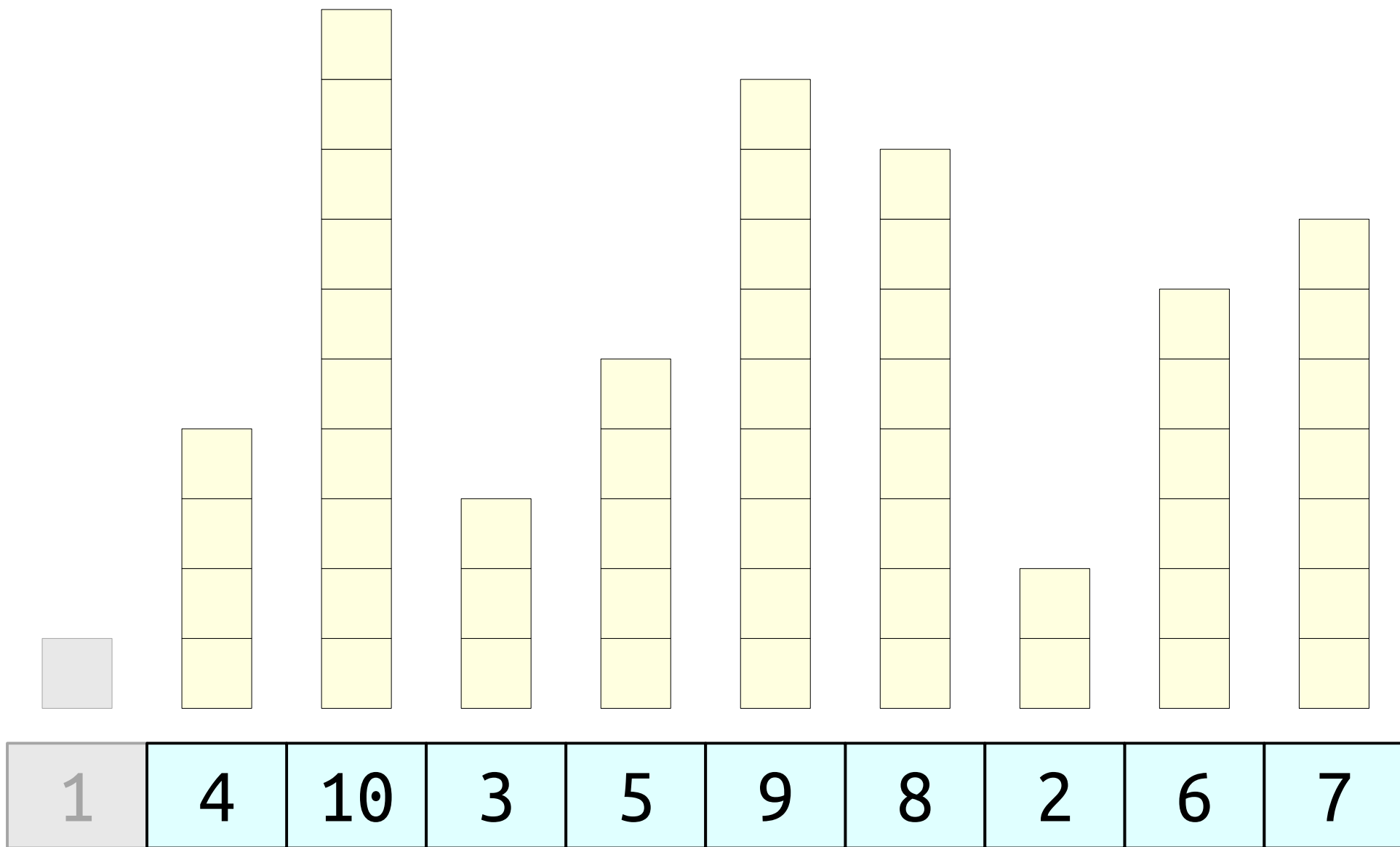
Sorting Algorithms

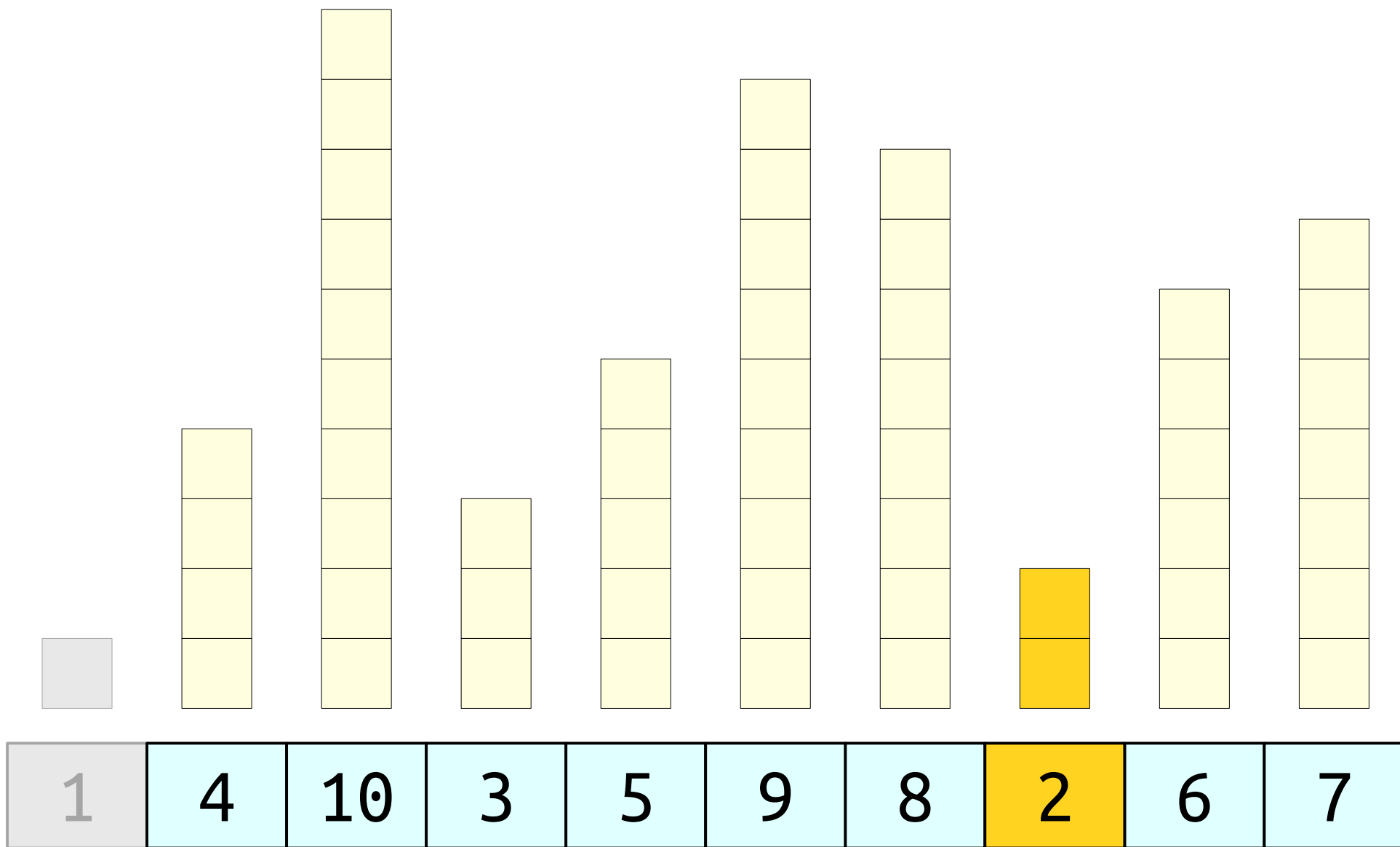
- A ***sorting algorithm*** is an algorithm that rearranges the terms of a sequence to put them in ascending order.
- Sorting is a supremely well-studied problem within computer science, and there are hundreds of ways to sort a list.
- We're going to cover the basics here; hit me up after class if you want to see how deep the rabbit hole goes.

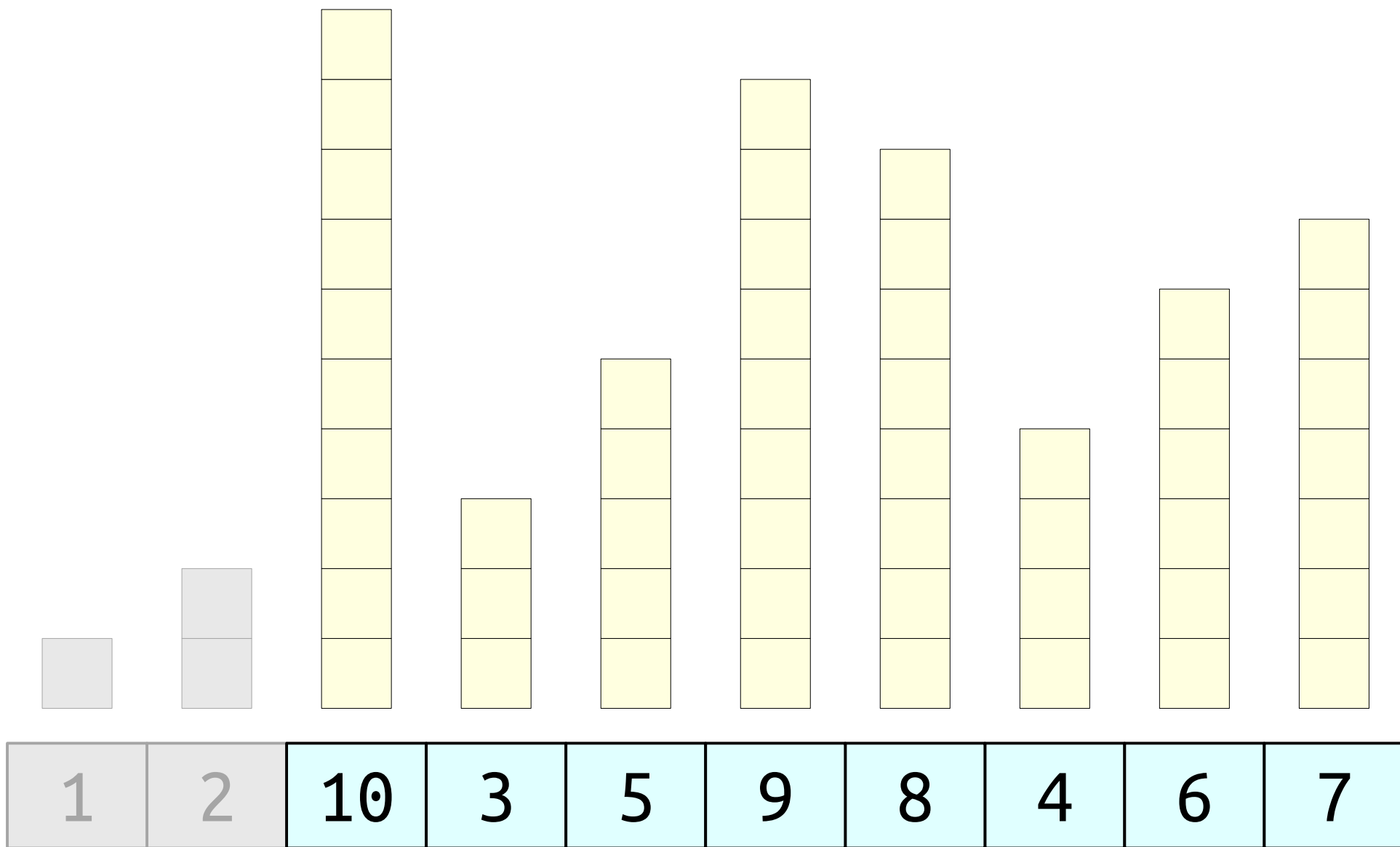
An Initial Idea: ***Selection Sort***

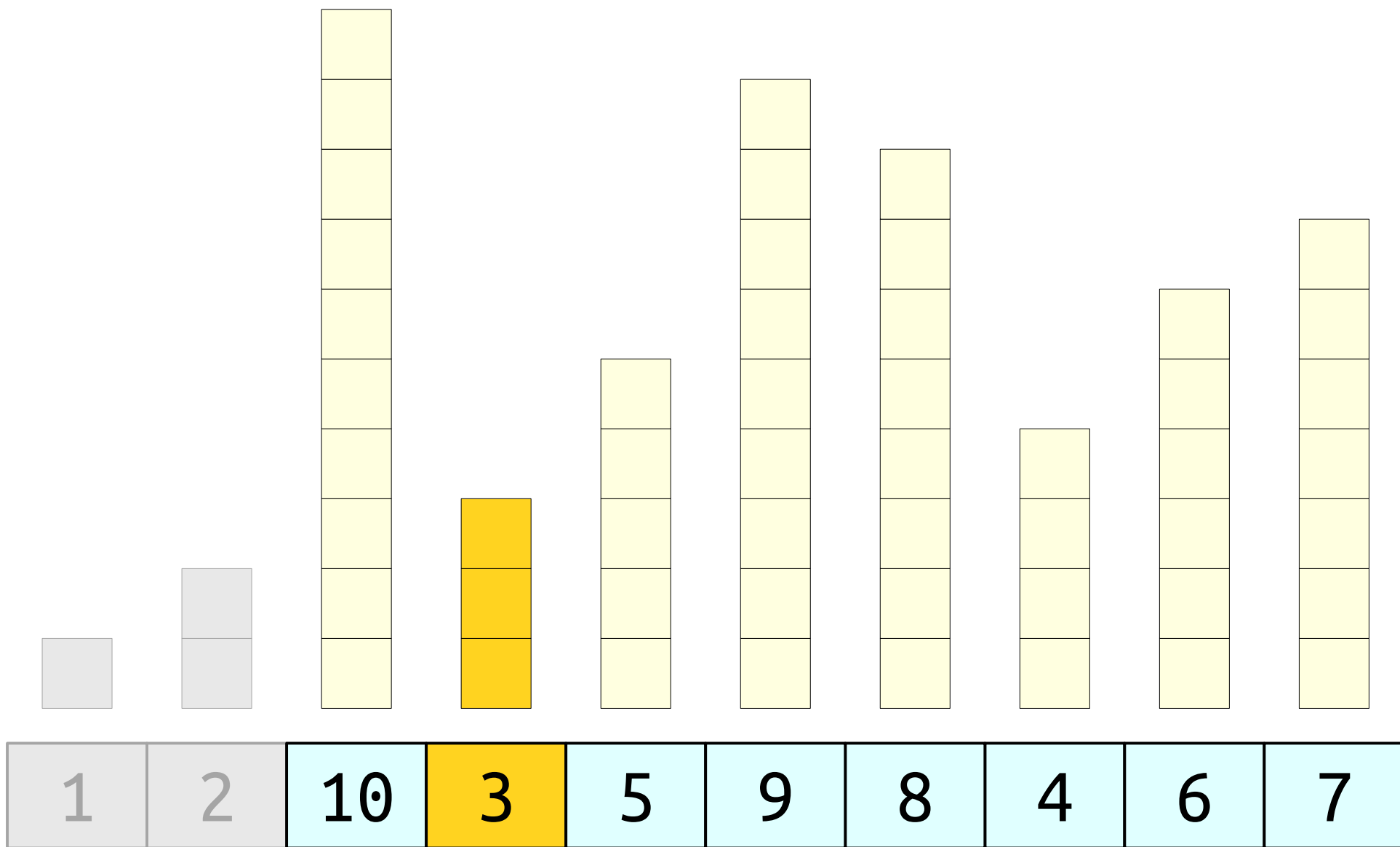


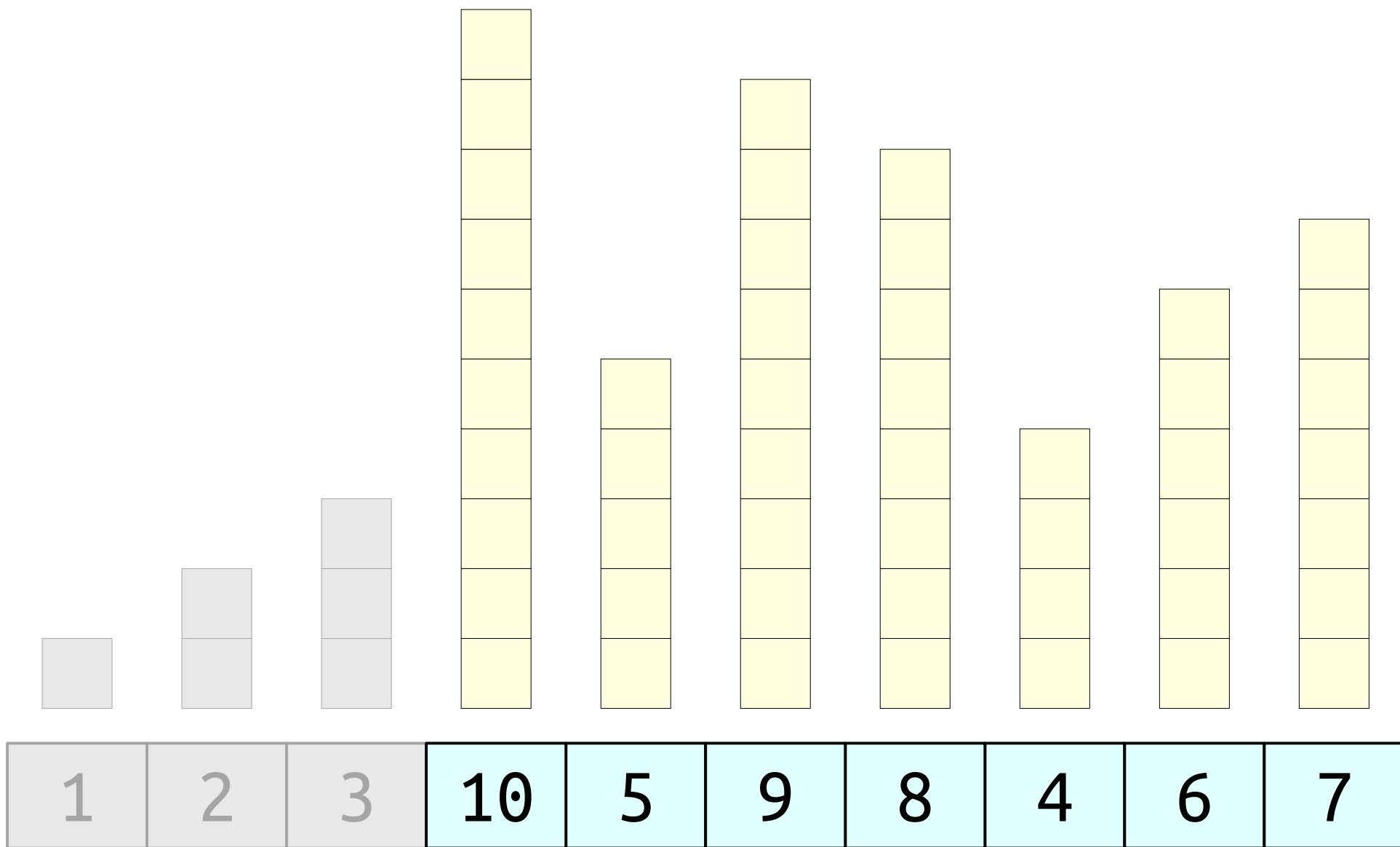


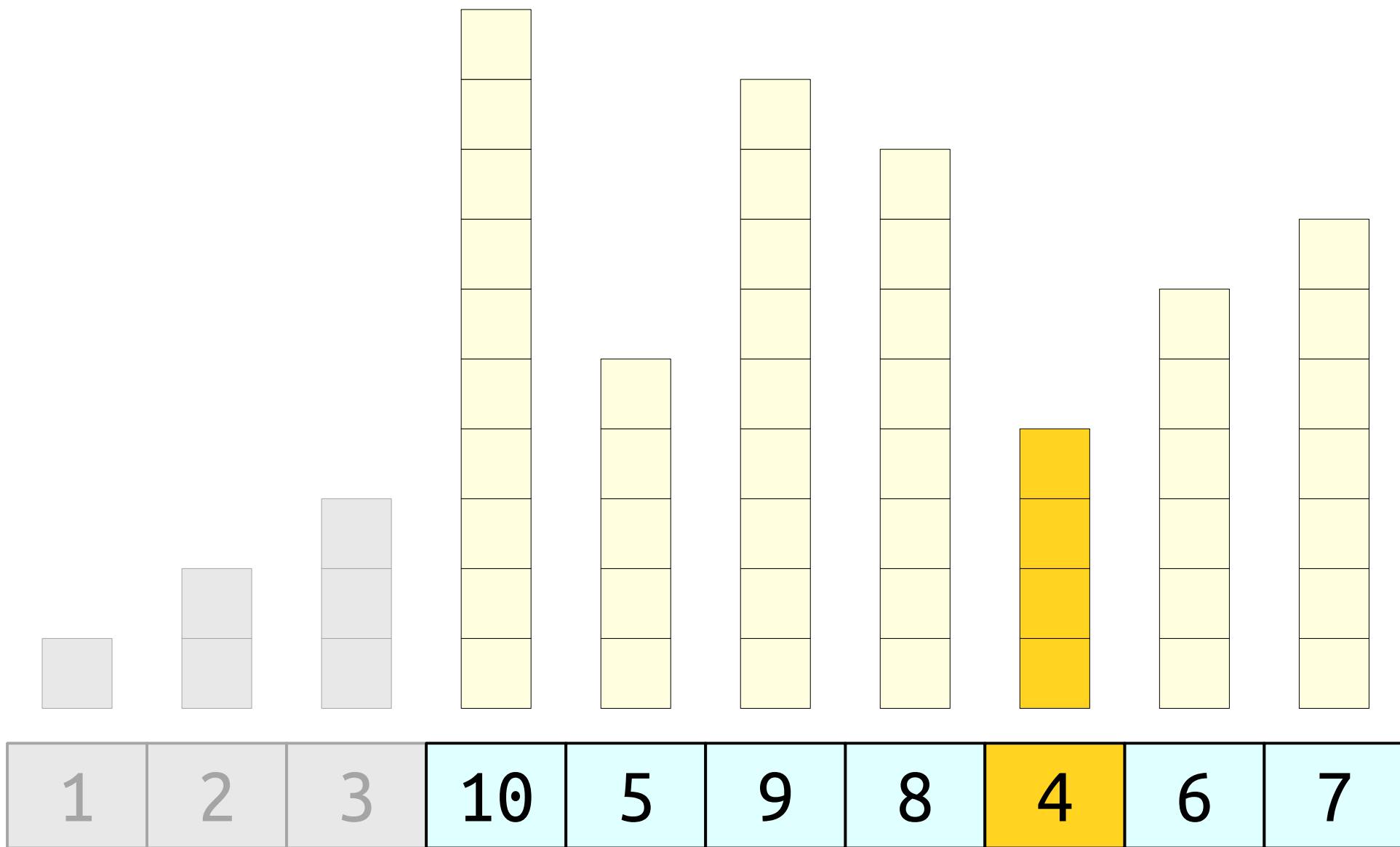


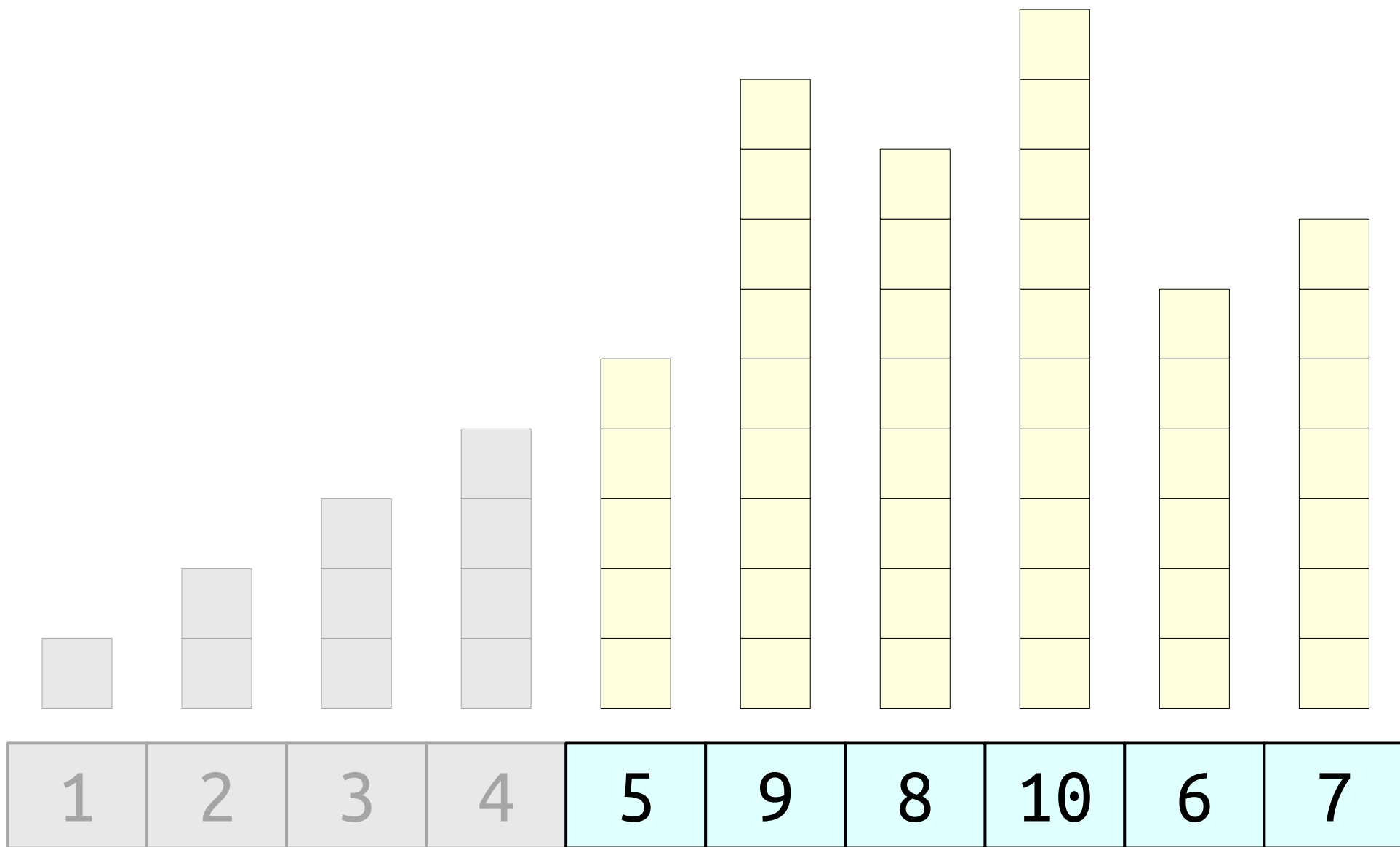


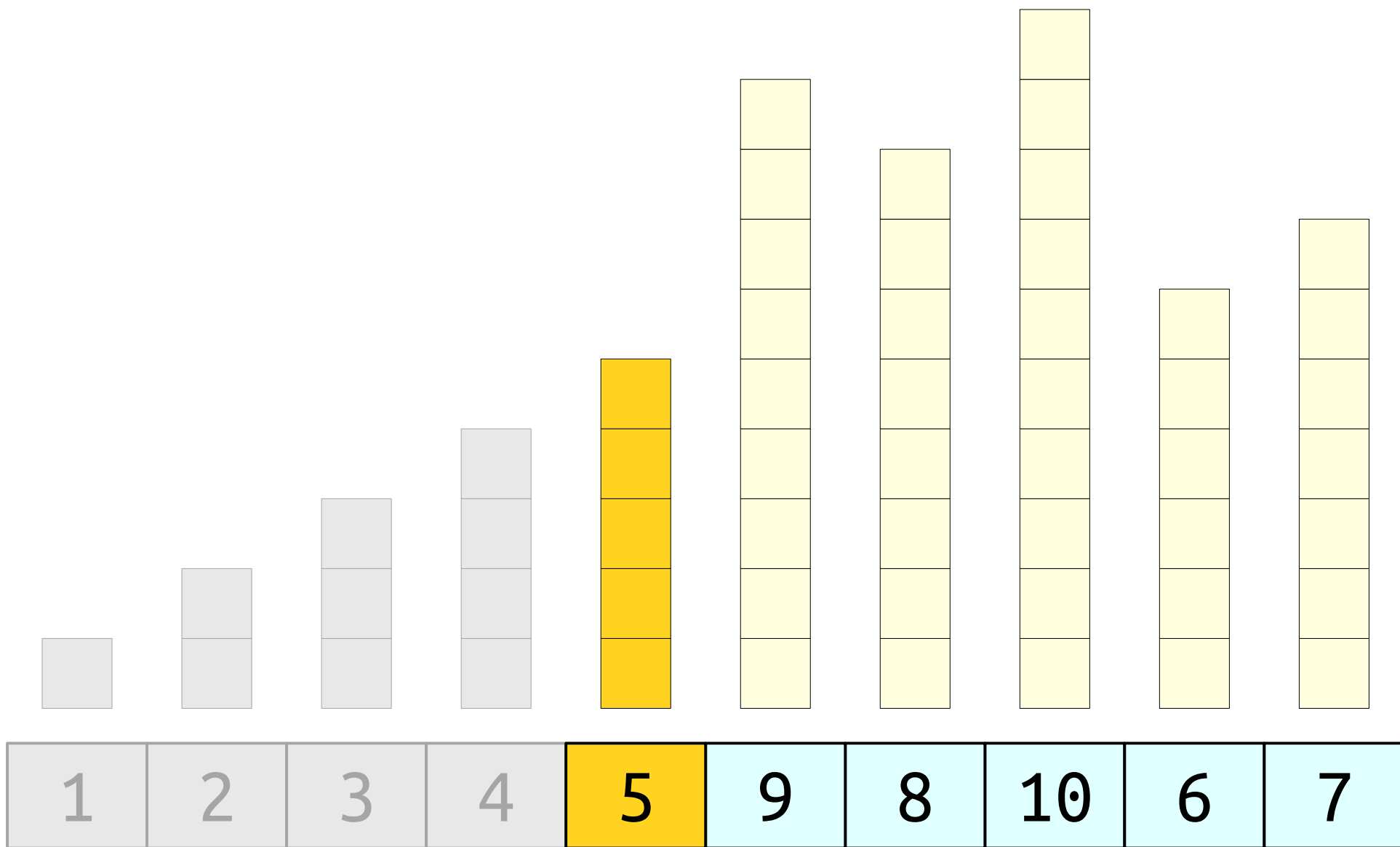


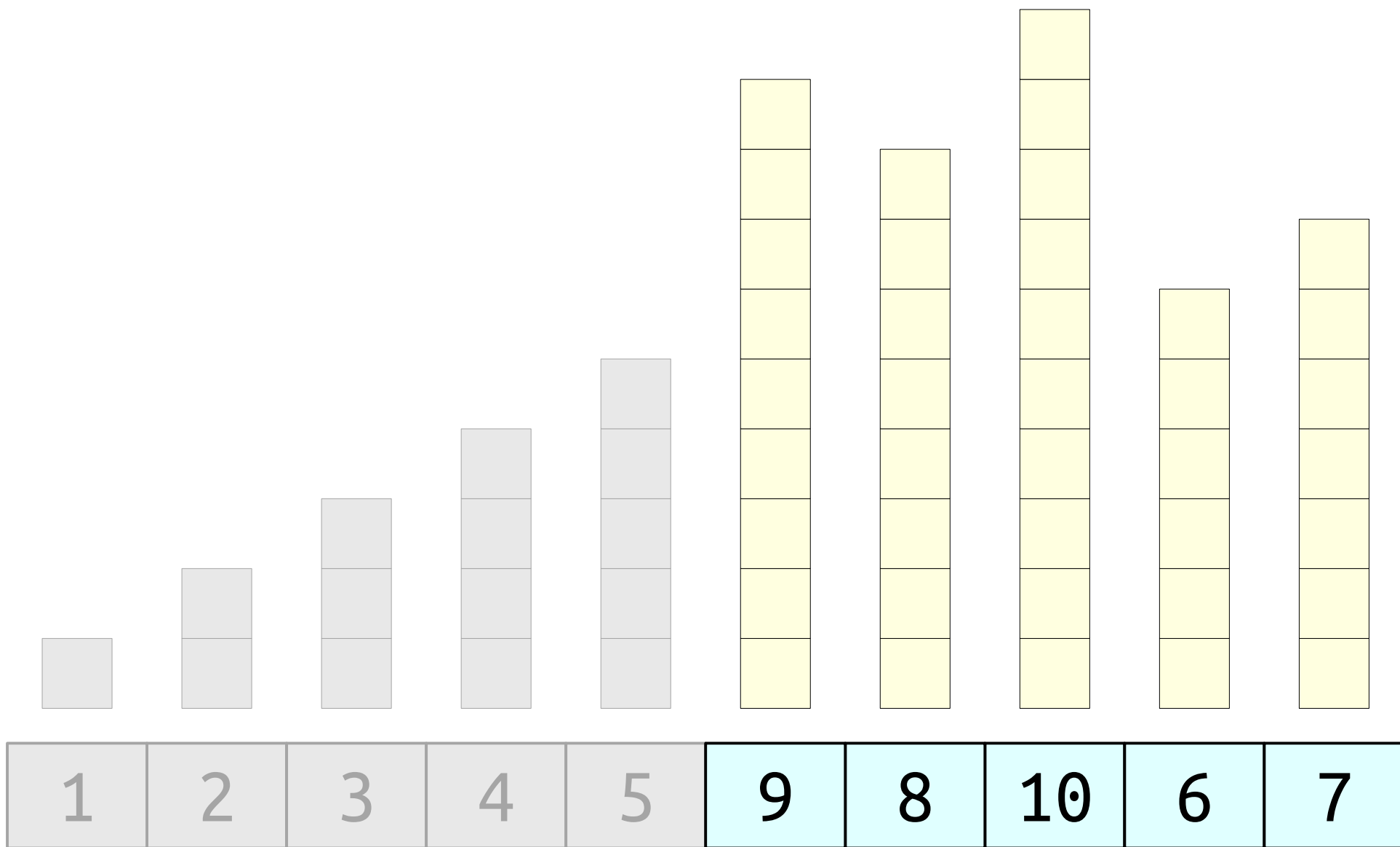


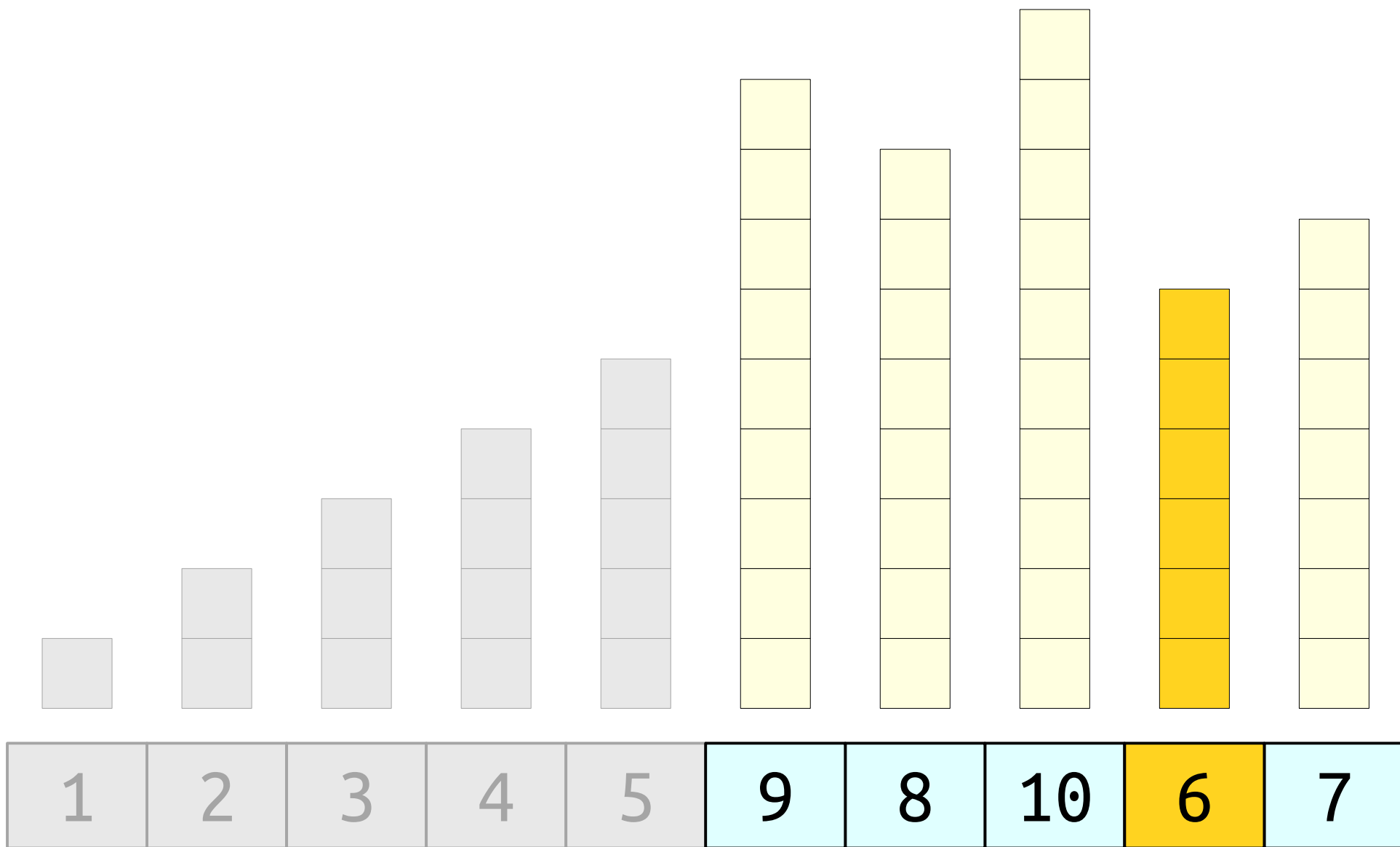


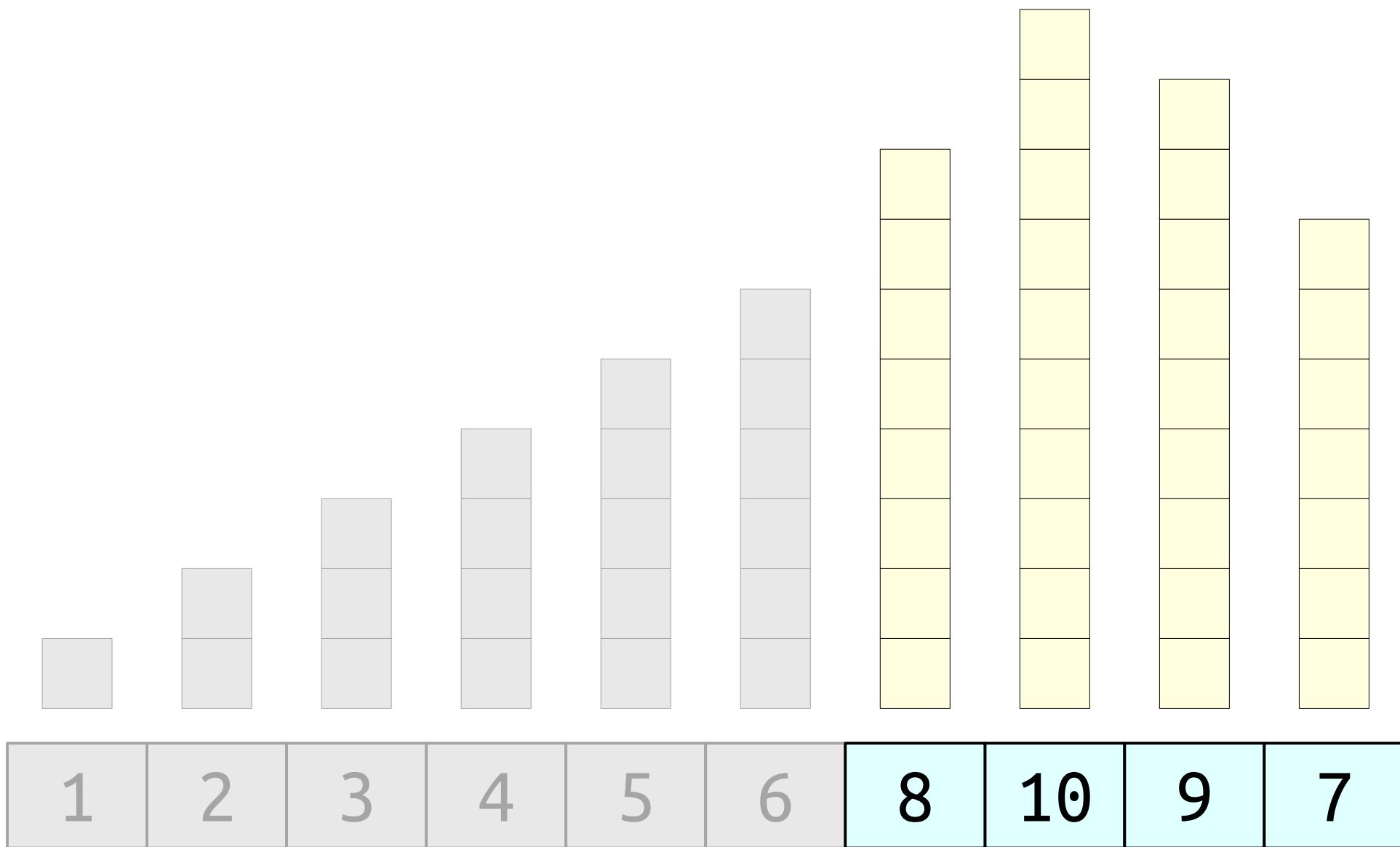


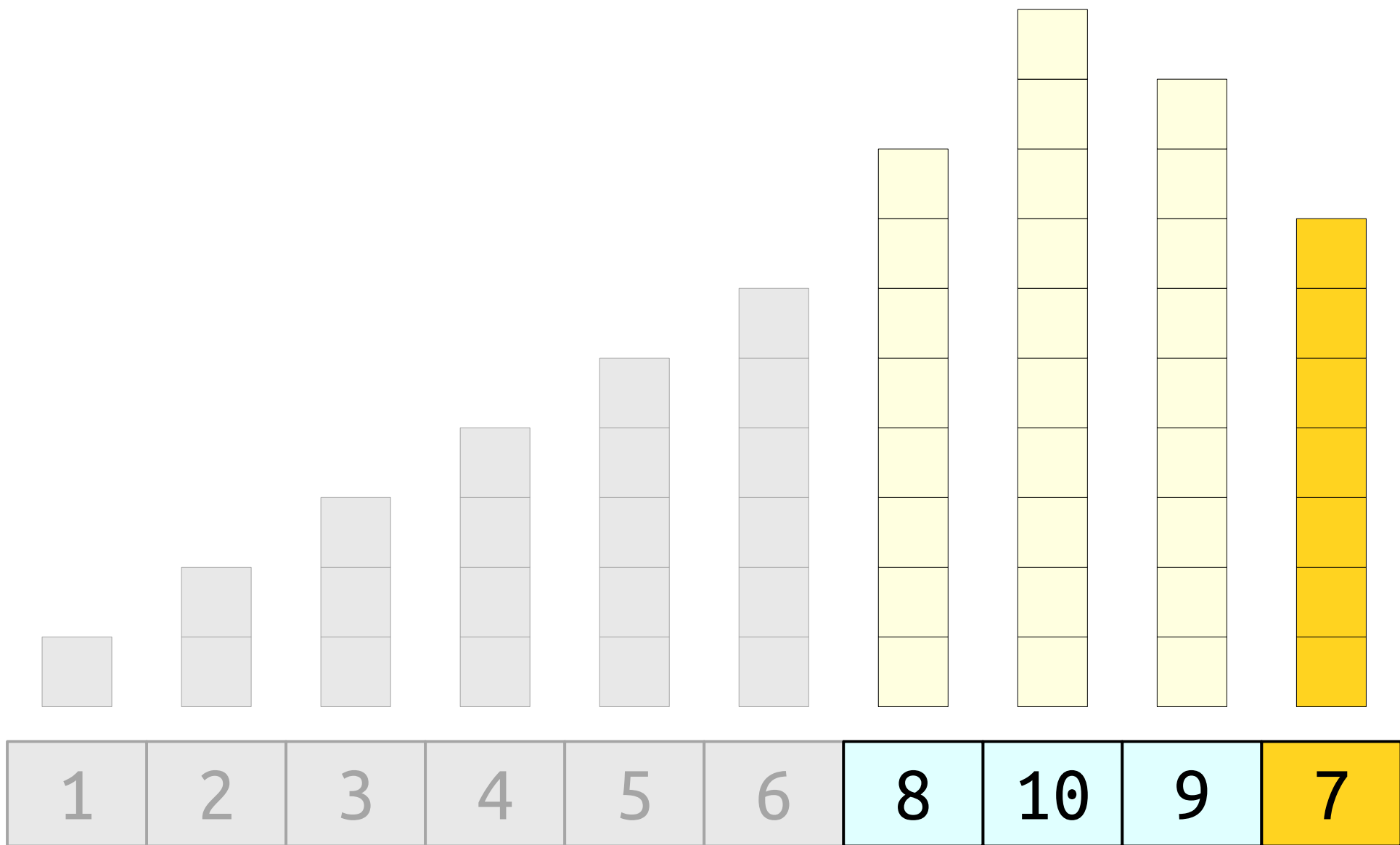


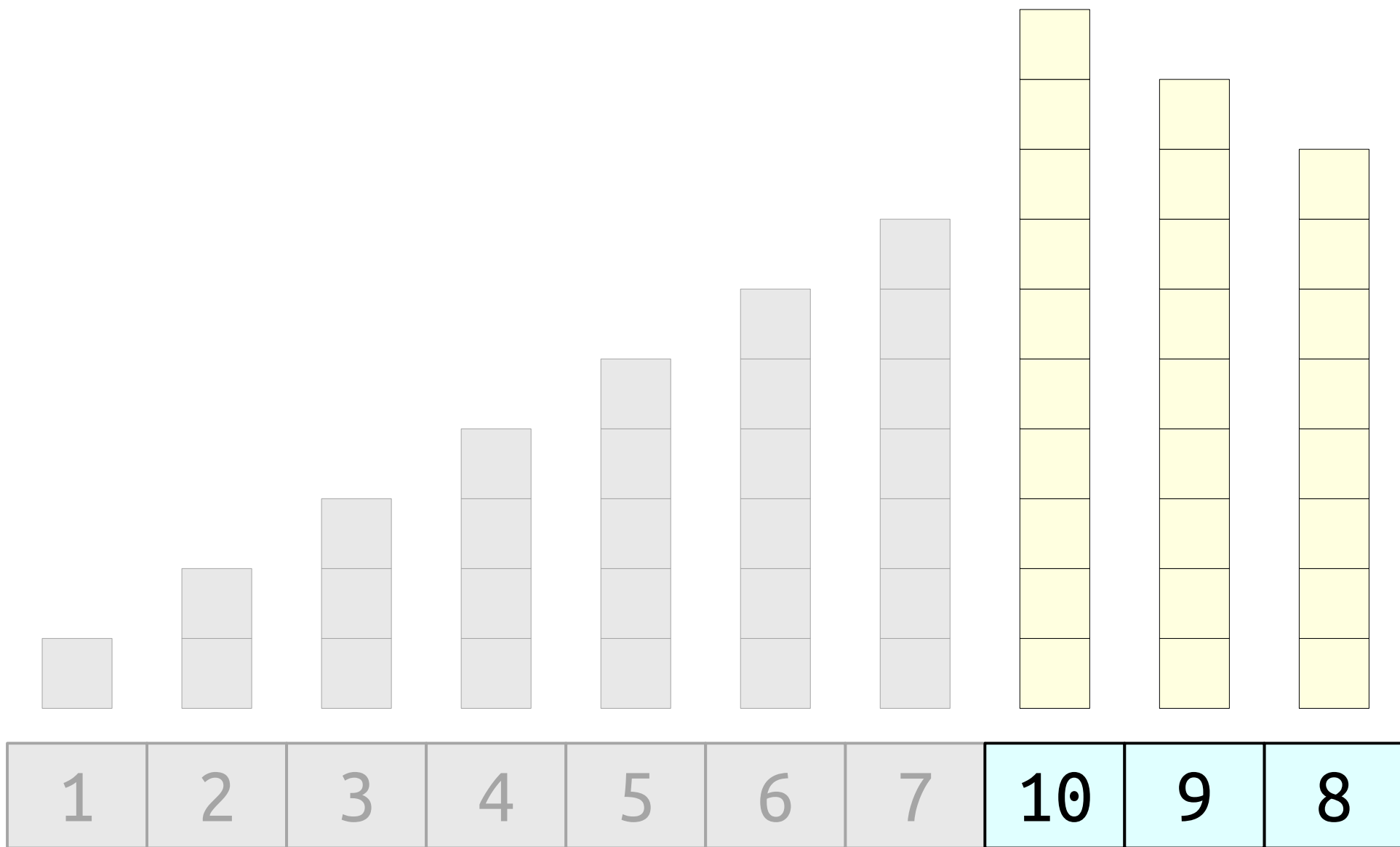


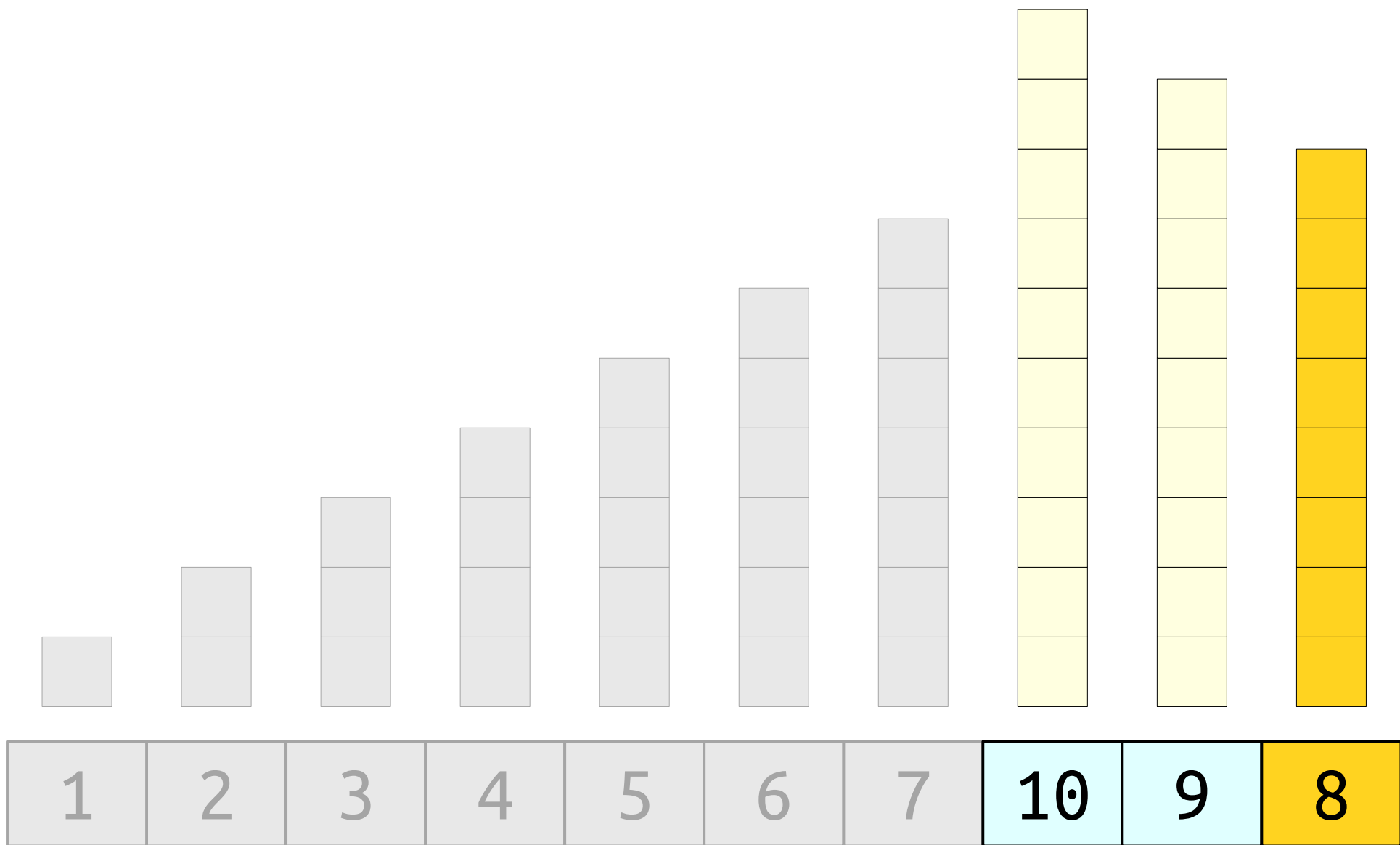


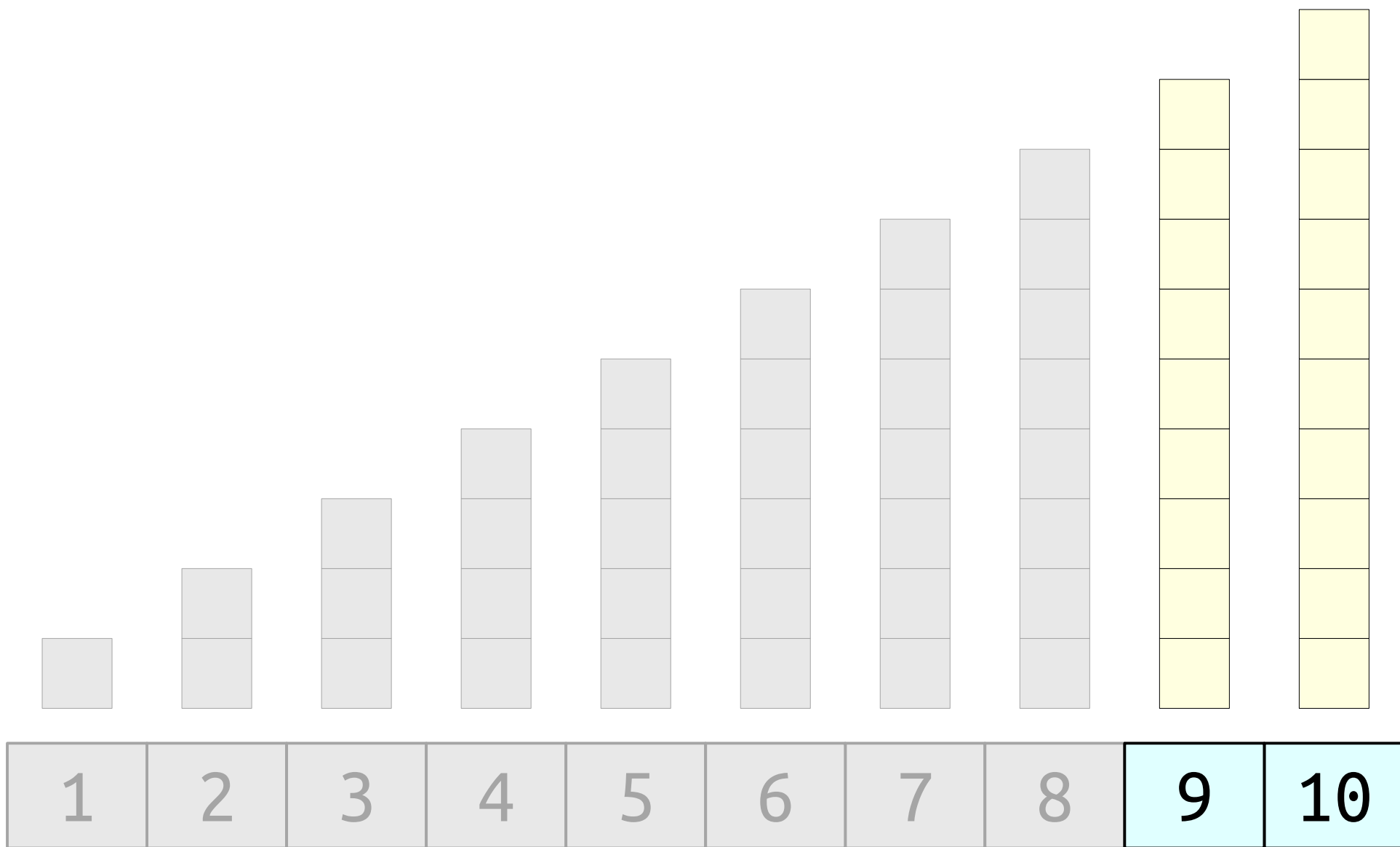


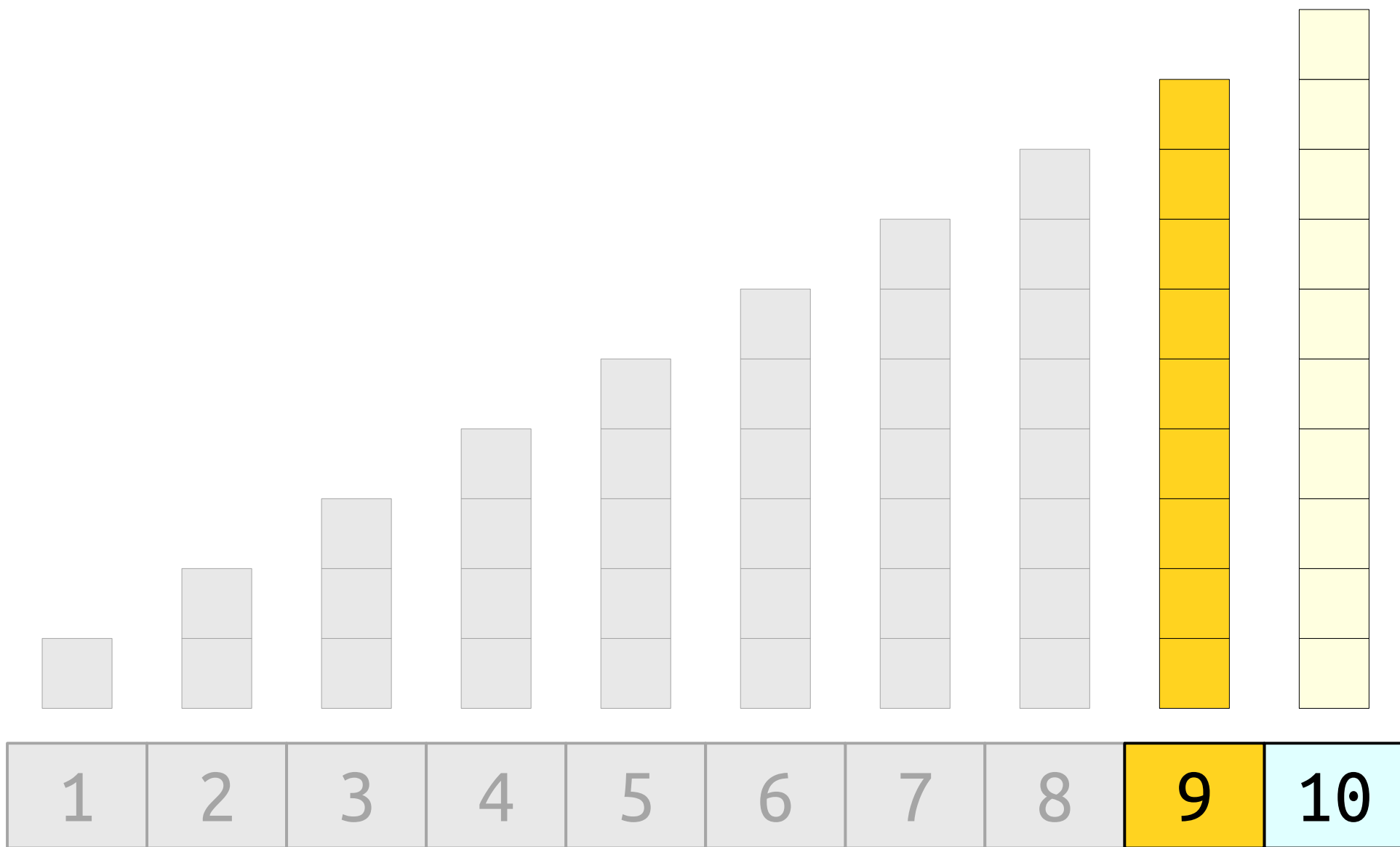


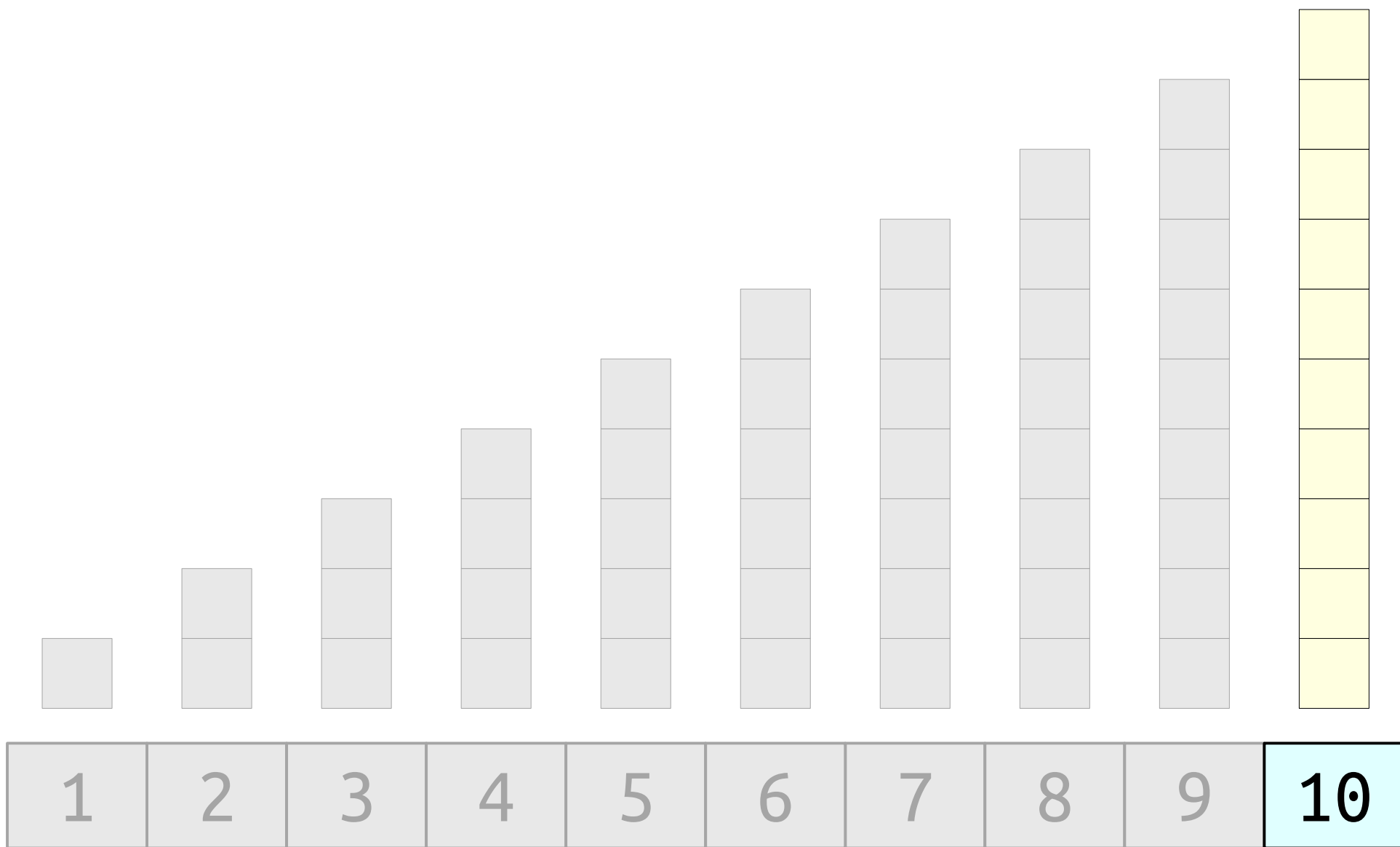


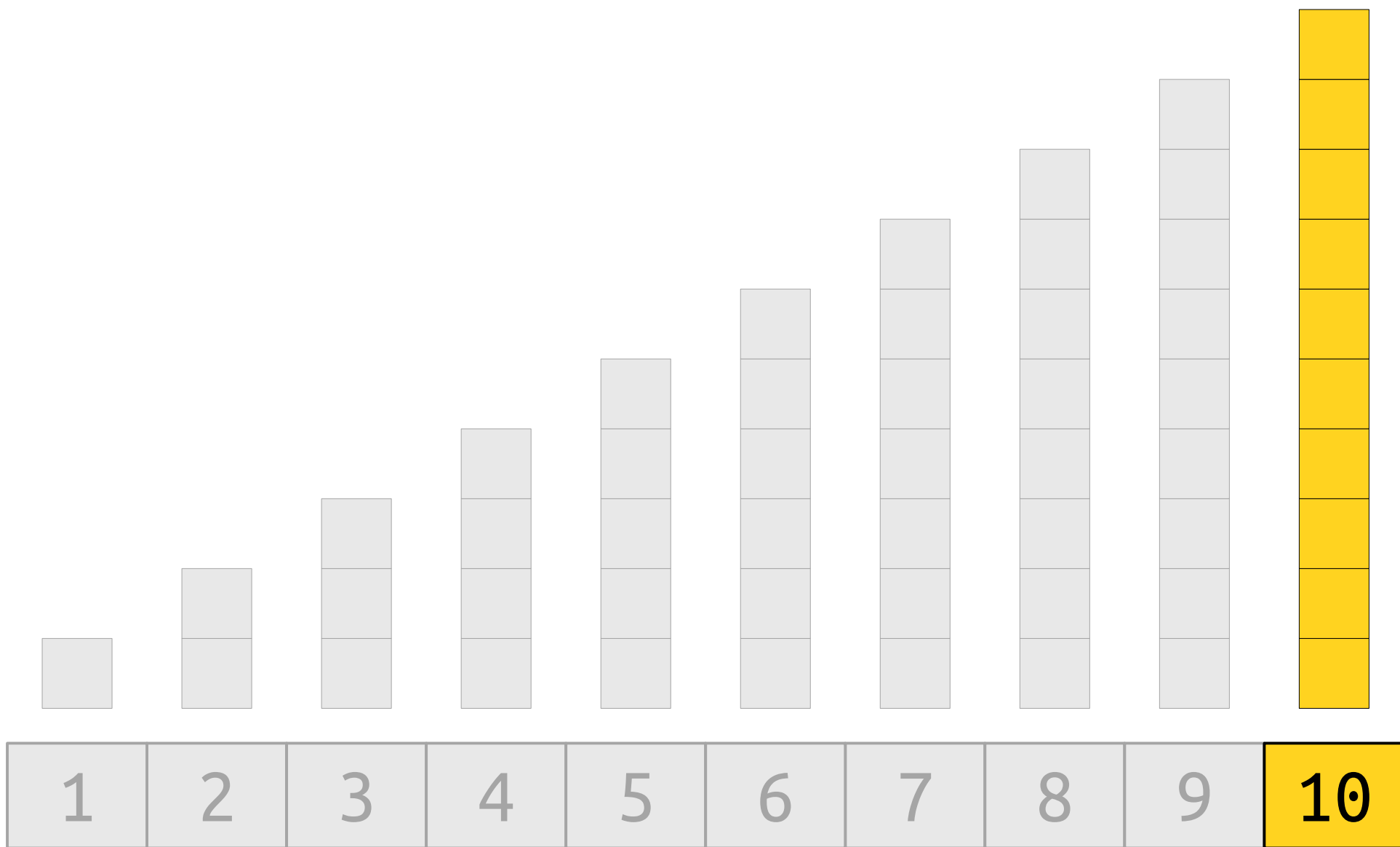


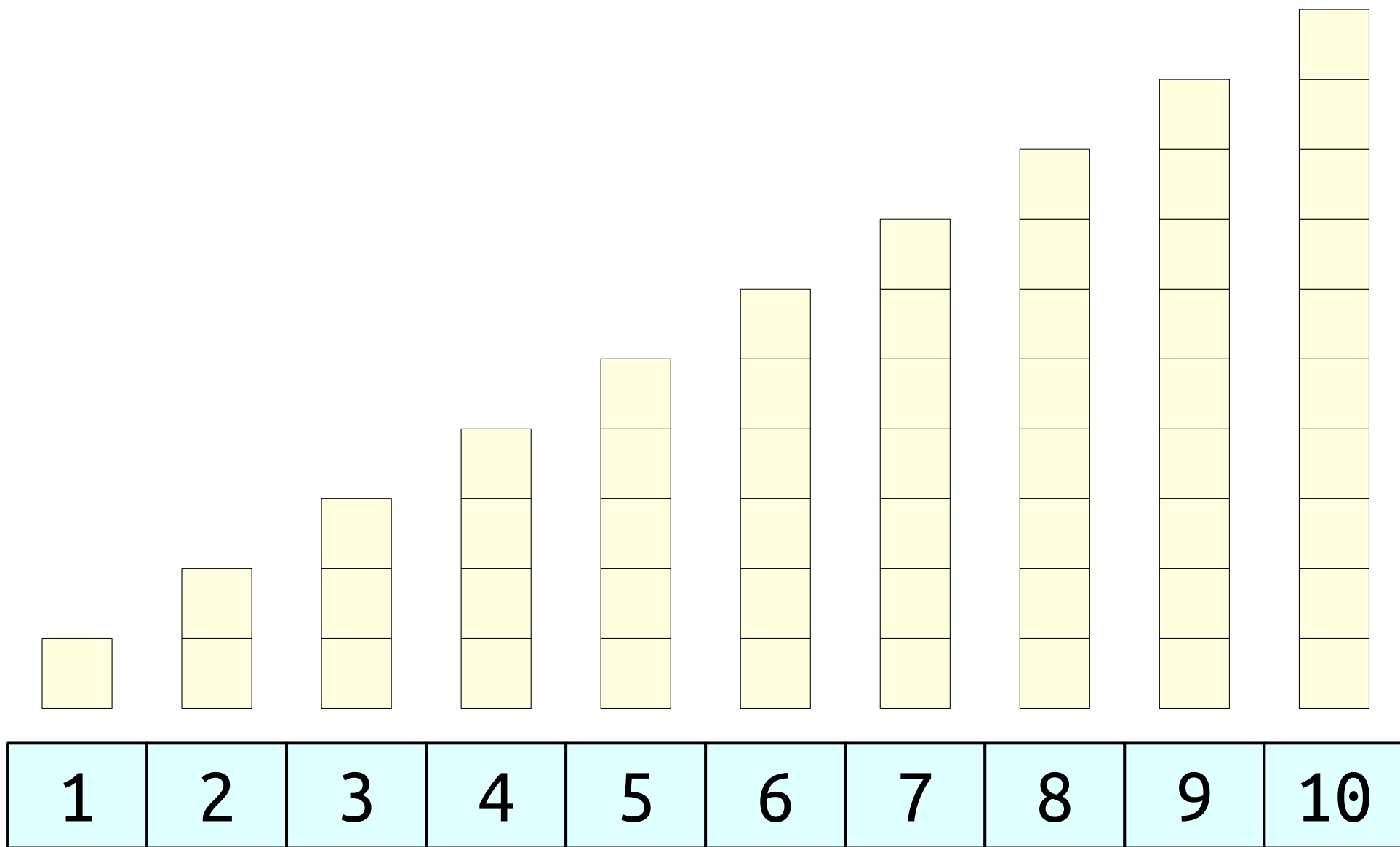












Selection Sort

- Find the smallest element and move it to the first position.
- Find the smallest element of what's left and move it to the second position.
- Find the smallest element of what's left and move it to the third position.
- Find the smallest element of what's left and move it to the fourth position.
- (etc.)
- Eventually all elements are put in the right place and the list is sorted.


```

void selectionSort(Vector<int>& elems) {
    for (int i = 0; i < elems.size(); i++) {
        /* Find the index of the smallest element occurring at position
         * i or later within the vector.
         */
        int smallestIndex = indexOfSmallest(elems, i);

        /* Move it to the current position. */
        swap(elems[i], elems[smallestIndex]);
    }
}

```

```

/* Given a vector and a starting point, returns the index of the
 * smallest element in that vector at or after the starting point.
 */
int indexOfSmallest(const Vector<int>& elems, int startPoint) {
    /* Your standard "look at a bunch of items and take the smallest"
     * loop; very similar to what you did in Rosetta Stone.
     */
    int smallestIndex = startPoint;
    for (int i = startPoint + 1; i < elems.size(); i++) {
        if (elems[i] < elems[smallestIndex]) {
            smallestIndex = i;
        }
    }
    return smallestIndex;
}

```

{ 46, 69, 20, 16, 09, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }



How fast is selection sort?

{ 46, 69, 20, 16, 09, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

How fast is selection sort?

{ 09, 69, 20, 16, 46, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

How fast is selection sort?

{ 09, 69, 20, 16, 46, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

How fast is selection sort?

{ 09, 69, 20, 16, 46, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

How fast is selection sort?

{ 09, 69, 20, 16, 46, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }



Finding the element that goes in position 0 requires us to scan all n elements.

How fast is selection sort?

{ 09, 69, 20, 16, 46, 10, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

↑

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 20, 16, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 16, 20, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 16, 20, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

How fast is selection sort?

{ 09, 10, 16, 20, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

Finding the element that goes in position 2 requires us to scan $n - 2$ elements.

How fast is selection sort?

{ 09, 10, 16, 20, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

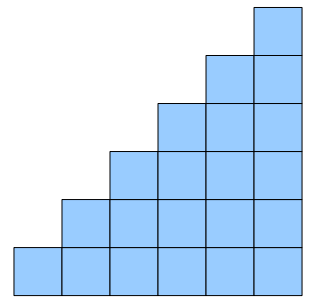
Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

Finding the element that goes in position 2 requires us to scan $n - 2$ elements.

...

Number of elements scanned:

$$n + (n-1) + (n-2) + \dots + 2 + 1.$$



How many boxes are in an n -layer stack of boxes like the one shown here? Give your answer using big-O notation.

Answer at <https://cs106b.stanford.edu/pollev>

{ 09, 10, 16, 20, 46, 69, 29, 90, 67, 18, 53, 20, 38, 20, 46 }

Finding the element that goes in position 0 requires us to scan all n elements.

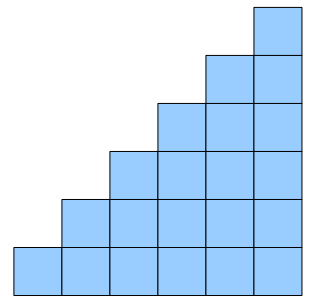
Finding the element that goes in position 1 requires us to scan $n - 1$ elements.

Finding the element that goes in position 2 requires us to scan $n - 2$ elements.

...

Number of elements scanned:

$$O(n^2)$$



How fast is selection sort?

Our theory predicts that the runtime of selection sort is $O(n^2)$.

Does that match what we see in practice?

What should we expect to see when we look at a runtime plot?

Time-Out for Announcements!

Midterm Exam Logistics

- Our midterm exam will be on **Monday, February 10th** from **7:00PM - 10:00PM**. Locations are assigned by last (family) name:
 - A – H: Go to Bishop Auditorium.
 - I – S: Go to Hewlett 200.
 - T – Z: Go to Hewlett 201.
- Exam format:
 - The exam covers L00 – L09 (basic C++ up through but not including recursive backtracking) and A0 – A3 (debugging through recursion).
 - It's a traditional sit-down, pencil-and-paper exam.
 - It's closed-book, closed-computer, and limited-note. You can bring an 8.5" × 11" sheet of notes with you to the exam. We will provide a syntax reference sheet for container types; it's up on the course website.
- We've posted a searchable bank of practice problems to the course website, along with three practice exams made of questions from that bank.
- We have reached out to everyone who we believe is taking an alternate exam with location / time info. If you haven't heard from us and you think you're taking an alternate exam, contact us ASAP.

Exam Prep Advice

- ***Our recommendation:***
 - Print out one of the practice exams and the reference sheet we provide.
 - Take the exam under realistic conditions – no computers, low-distraction settings, have a timer running, etc.
 - Have your SL (or an SL in the LaIR) look over your answers and offer detailed feedback on what worked and what didn't.
 - Focus your studying on the areas you need more practice with.
 - Adjust your notes sheet accordingly.
- Yes, this takes time. But it's *vastly* more effective than most other study strategies.

Midterm Review Session

- The amazing SL team will be holding a midterm review session this week:

Wednesday, February 5th

5:00PM - 7:00PM

Location TBA

- Come with questions, leave with answers!

Next Time

- ***Mergesort***
 - A beautiful, elegant sorting algorithm.
- ***Analyzing Mergesort***
 - An unusual runtime analysis.